

Autonomous SOC

Containerized Cyber Range

Master Technical Documentation

Architecture · Machine Learning Pipeline · Agentic Orchestration · Engineering Journal

A reference for understanding and presenting every part of this system - written to be read end to end by a teammate with no prior exposure to the codebase.

Table of Contents

- 1. Introduction & Project Overview** — what this system is, why it exists, key terms
- 2. System Architecture** — network topology, containers, data flow
- 3. The Attacker Node** — synthetic and real attack generation
- 4. The SOC Sensor Brain** — packet capture, ML inference, orchestration, API, storage
- 5. The Triage & Resolution Flow** — how an alert becomes a decision
- 6. The Machine Learning Pipeline** — data, features, training, calibration, evaluation
- 7. The Dashboard UI** — every view, what it shows, how it's wired
- 8. Guardrails & Safety Mechanisms** — consolidated reference
- 9. Debugging Journal & Lessons Learned** — real bugs found and fixed this project
- 10. API Reference** — every endpoint
- 11. Known Limitations & Future Work**
- 12. Appendix** — file manifest, environment variables, common commands

1. Introduction & Project Overview

1.1 What this project is

This project is a fully self-contained, Docker-based cyber range: a miniature enterprise network with a DMZ and an internal segment, an attacker that can launch real and simulated attacks against it, and an **autonomous Security Operations Center (SOC)** that watches all traffic crossing between the two zones, decides what's malicious using a two-stage machine learning classifier, and then hands genuine anomalies to an AI agent pipeline (built on LangGraph, powered by Google's Gemini) that triages the threat, looks up business context about the target asset, and decides whether to autonomously block the attacker at the network layer - or, for the most sensitive assets, defer that decision to a human.

Nothing here is simulated at the level of "pretend attacks happen." Real packets cross a real Docker bridge network, a real Random Forest model classifies them, real `iptables` rules get written to the Linux kernel, and (when not in cost-saving dry-run mode) a real call goes out to Gemini and comes back with real reasoning about the flow.

1.2 Why it exists

The project demonstrates an end-to-end applied security engineering pipeline: traditional ML for high-throughput, low-latency detection (a neural network would be too slow to run on every network flow in real time) combined with generative AI for the parts that benefit from reasoning and natural-language judgment - triage narrative, business-impact assessment, and the containment decision itself. It is built to be defensible under questioning: every claim in this document is backed by something that was actually run and observed during development, not by what the code was merely intended to do.

1.3 Key terms glossary

Term	Meaning in this project
SOC	Security Operations Center - the team/system responsible for detecting and responding to security threats. Here, an autonomous software stand-in for that team.
NIDS	Network Intrusion Detection System - software that inspects network traffic for malicious patterns. <code>live_sensor.py</code> is this project's NIDS.
Network flow	A bidirectional sequence of packets between the same two endpoints (IP + port pairs) within a time window. <code>NFStream</code> aggregates raw packets into flows; the ML models classify one flow at a time, not one packet at a time.

Random Forest	An ensemble of decision trees, each trained on a random subset of data/features; the final prediction is a majority vote. Chosen here over a neural net for inference speed and resilience to noisy features - important when classifying flows in real time.
LangGraph	A framework for building multi-step, stateful AI agent workflows as a graph of nodes (here: Triage → Context → Response) where each node can read/write shared state and optionally call tools.
Agentic tool-calling	Letting an LLM decide whether and how to invoke a function (a "tool") rather than a human/programmer deciding for it. This project binds <code>`block_ip_tool`</code> and <code>`lookup_asset_tool`</code> so the model itself chooses to act.
TTL	Time-to-live - here, how long an automatic network block stays active before it automatically expires and is reviewed again.
Allow-list	A list of addresses the system will never take action against, regardless of what an alert or an AI model recommends - the last line of defense against a wrong or manipulated decision.
Human-in-the-loop	A design pattern where the AI proposes an action but a human must approve it before it executes, reserved here for the highest-value assets.

1.4 What "autonomous" actually means here

This is worth being precise about, since it comes up under questioning: the system is autonomous in the sense that *no human needs to be in the loop for it to detect and contain a threat* - but it is not unconditionally autonomous. Three separate guardrails (an allow-list, a rate limit, and a criticality-based approval requirement) can each independently override the AI's decision, and a human can always intervene via the dashboard. Section 8 covers this in full; it is one of the more defensible design points in the whole project, and the one most worth understanding deeply if presenting this work.

2. System Architecture

2.1 Network topology

The environment is split into two isolated Docker bridge networks, joined only through the SOC sensor, which acts as the router between them (this is enforced by the sensor container being the only one attached to both networks, and by the sensor running with `net.ipv4.ip_forward=1` and holding the ``NET_ADMIN`` capability needed to write firewall rules):

Zone	Subnet	Hosts
DMZ (<code>`dmz_net`</code>)	10.5.1.0/24	attacker-node (10.5.1.10), dmz-web (10.5.1.20, nginx), sensor's DMZ interface (10.5.1.254)
Internal (<code>`internal_net`</code>)	10.5.2.0/24	internal-db (10.5.2.20, Redis), hr-portal (10.5.2.30, mock HTTP app), sensor's internal interface (10.5.2.254)

All traffic from the simulated attacker to the internal network must physically cross the sensor, which is what makes a "sit in the middle and inspect everything" NIDS architecture possible without special mirror ports or taps - the standard set up in a real enterprise, replicated here with plain Docker networking.

2.2 Containers (from `docker-compose.yml`)

Service	Image / build	Role
attacker-node	alpine:3.18 + installed tools	Forges synthetic attack traffic (Scapy) or drives real tools (nmap, hping3) against internal targets.
dmz-web	nginx:alpine	A vulnerable-looking external-facing decoy web server.
internal-db	redis:alpine	Represents a sensitive internal data store; registered as <code>`Critical`</code> criticality.
hr-portal	python:3.10-alpine, <code>`http.server`</code>	Represents a sensitive internal app; registered as <code>`High`</code> criticality.
soc-sensor-brain	custom build (Dockerfile.sensor)	Privileged router + ML classifier + LangGraph orchestrator + Flask API. The core of the system.

dashboard-ui	custom build (Dockerfile.dashboard)	React + TypeScript frontend, served by Vite, polling the sensor's API every 1.5s.
--------------	-------------------------------------	---

The sensor container runs with `privileged: true` and mounts `./soc_brain` as a live volume, so code changes on the host take effect on container restart without a rebuild. The dashboard's Dockerfile instead does a build-time `COPY` - a real gotcha encountered this project (Section 9) - so frontend changes require `docker compose up --build dashboard-ui` to actually take effect.

2.3 End-to-end data flow

1. The attacker node forges or drives traffic toward an internal target.
2. The sensor's NFStreamer instances (one per interface, `eth0=DMZ`, `eth1=internal`) capture packets and assemble them into bidirectional flows.
3. Each completed flow is fed through the **binary Random Forest** (Normal vs. Attack). Flows the binary model calls Normal are separately checked by a lightweight **port-scan correlator** that looks across flows, not within one, since a real port scan doesn't look anomalous one flow at a time.
4. Flows flagged Attack go through the **multi-class Random Forest** to assign a specific category (DoS, Fuzzers, Exploits, Backdoor, Reconnaissance), and - if binary confidence ≥ 0.55 - an alert is created and handed to the LangGraph orchestrator on its own background thread.
5. The orchestrator runs **Triage** → **Context** → **Response**. Response either blocks the source IP outright, queues it for human approval, or takes no action, depending on the guardrails in Section 8.
6. Every step along the way writes to an in-memory event store, which the dashboard polls and renders live.

3. The Attacker Node

`attacker/attacker_api.py` is a small Flask service running inside the DMZ. It exposes attack control over HTTP so the dashboard (and, indirectly, a person clicking buttons) can start/stop attacks without shelling into the container.

3.1 Synthetic attack engine

The original and still-default mechanism crafts raw Ethernet frames directly with Scapy - bypassing the OS TCP/IP stack entirely - tuned to hit exact packet-count, byte-size, and duration targets per attack type, so the resulting NFStream-observed flow statistics land inside a class the ML model was calibrated to recognize:

Attack type	Protocol	Src/Dst packets	Mean packet size (src/dst)	Duration
dos	UDP	20 / 18	174 / 121 bytes	1.88s
backdoors	UDP	7 / 2	103 / 18 bytes	2.6s
fuzzers	TCP	10 / 6	91 / 45 bytes	0.21s
exploits	TCP	10 / 8	100 / 44 bytes	0.84s
port_sweep	TCP	10 / 6	84 / 45 bytes	0.18s

Why subtract header lengths from the payload: NFStream measures full Layer-2 frames, but Scapy's payload argument only controls the application-layer bytes. The code subtracts a 14-byte Ethernet header and a 20 or 28-byte TCP/UDP+IP header before constructing each packet, so the *observed* mean packet size on the wire matches the calibrated target exactly rather than overshooting it by the header size.

3.2 Real attack tools

Two attack types have a validated, working **real-tool** path, toggled by a `real_tools` flag from the dashboard:

- **Volumetric DoS** - real `hping3`, bounded to 500 packets per burst rather than an unbounded flood (an unbounded `--flood` was tested and never produced a completed, classifiable NFStream flow at all within observation - see Section 9). Confirmed to reliably classify as DoS at 55-95% confidence.
- **Stealth Port Sweep** - real `nmap -sT`, looped continuously. A genuine multi-port scan fragments into many separate 1-2 packet flows, each individually indistinguishable from an ordinary short connection to the per-flow classifier - this is caught by the sensor's cross-flow port-scan correlator instead (Section 4.1), not by the RF models.

Exploits, Fuzzers, and Backdoors remain on the synthetic engine even with the toggle enabled - real `hping3`/TCP-handshake traffic for these was tested and produces a genuinely different statistical footprint (real

SYN/ACK/RST sequences vs. hand-crafted packets with no OS-level handshake) that the current calibration doesn't recognize reliably. This is stated in the UI itself, not hidden.

3.3 Endpoints

Route	Method	Purpose
/attack	POST	Start an attack: {type, target, source_ip, randomize_ip, real_tools}
/stop_attack	POST	Stop one attack type by name
/stop	POST	Stop everything, kill any lingering hping3/nmap processes
/logs, /clear_logs	GET / POST	Attacker-side log tail, used by the dashboard's Debug Logs view

4. The SOC Sensor Brain

Everything under `soc_brain/` runs inside the single privileged `soc-sensor-brain` container. This is the most important part of the codebase, and the one worth understanding file by file.

4.1 `live_sensor.py` - packet capture, feature extraction, ML inference

Spawns one background thread per network interface, each running its own `NFStreamer` instance. For every completed flow (once it clears `NFStream`'s 10-second active timeout or 5-second idle timeout), the sensor:

1. Discards it if it predates the last environment reset, or if its source IP isn't inside the `10.5.` range (filters out ambient host/Docker-bridge noise).
2. Computes 9 derived features (see Section 6.2) from the raw `NFStream` flow object.
3. Runs the binary classifier. If it says `Attack` with confidence ≥ 0.55 , runs the multi-class classifier and creates an alert.
4. If the binary classifier says `Normal`, additionally checks the flow against the **port-scan correlator** before moving on.

The port-scan correlator

Why this exists: a real port scan is many separate 1-2 packet flows, one per probed port - each one looks statistically identical to an ordinary short connection to a classifier that only ever sees one flow at a time. The scan signature ("many destination ports touched by one source in a short window") only exists *across* flows. This function keeps a short rolling window (5 seconds) per source IP of recently-touched (`dst_ip`, `dst_port`) pairs; once a source crosses 5 distinct targets with small flows (≤ 4 packets each), it fires a synthetic Reconnaissance alert directly - the same technique real NIDS engines like Zeek and Suricata use for scan detection, applied independently of the RF models. This was added specifically because a real `nmap` scan was going completely undetected before it existed (Section 9.3).

```
SCAN_WINDOW_SECONDS = 5
SCAN_DISTINCT_TARGET_THRESHOLD = 5
SCAN_MAX_FLOW_PACKETS = 4

def check_port_scan(src_ip, dst_ip, dst_port, total_packets):
    if total_packets > SCAN_MAX_FLOW_PACKETS:
        return None
    # ... track (dst_ip, dst_port, timestamp) per src_ip in a deque,
    # drop entries older than the window, and fire once >= 5 distinct
    # targets are seen (with a 30s suppression window per source so one
    # burst doesn't re-fire on every subsequent probe)
```

4.2 `train_model.py` - data engineering and model training

Covered in full depth in Section 6. In summary: downloads UNSW-NB15 if not present, engineers 9 features, injects jittered synthetic "anchor" rows so the demo attacker's forged traffic reliably lands inside a recognized class, then trains the binary and multi-class Random Forests with `n_estimators=150`, `max_depth=15`, `class_weight='balanced'`.

4.3 orchestrator.py - the LangGraph agent pipeline and every guardrail

This is the largest and most consequential file (424 lines). It defines:

- The three-node LangGraph state machine: `triage` → `context` → `response` → `END`.
- Two agent tools bound to the LLM: `lookup_asset_tool` (Context node) and a per-call `block_ip_tool` built fresh for each alert via `make_block_ip_tool()` (Response node).
- Every safety guardrail: the protected allow-list, the rate limiter, tiered human-approval, and TTL-based block expiry. All covered together in Section 8, since they only make sense as a system.
- `trigger_swarm()`, the entry point called by `live_sensor.py` for every alert, which gates execution on a per-source-IP cooldown before invoking the graph.

Why `block_ip_tool` is built fresh per call, not once at import time

A real concurrency bug found and fixed during this project: the first implementation passed alert context (which asset is being targeted, and therefore whether approval is required) into the tool via `threading.local()`, on the assumption that each alert's tool call would run on the same thread that set up the context. It doesn't - LangGraph's `ToolNode` can execute the tool on a different worker thread than the one that invoked the agent, so the thread-local value was silently empty by the time the tool ran, and the approval gate defaulted to "unknown criticality" and skipped straight to auto-block. The fix replaces thread-local storage with a closure: `make_block_ip_tool(alert_data, ...)` returns a new tool function that captures its context by Python closure semantics, which work correctly regardless of which thread eventually calls it. This was caught by a test that asserted a High-criticality target routes to approval, not direct block - it failed with thread-local storage and passed with the closure, which is exactly the kind of test that's worth being able to explain if asked "how do you know your concurrency model is correct."

4.4 dashboard_api.py - the REST API

A Flask app run in a background thread inside the same process as the sensor, so it shares the in-memory event store directly with no IPC needed. Full endpoint list in Section 10.

4.5 event_store.py - shared in-memory state

A single thread-safe class (`EventStore`, guarded by one `threading.Lock`) holding four things: alerts (capped at 100), mitigations (capped at 100), flow events (capped at 50), and a `last_reset_time` timestamp used to discard stale flows captured before the last reset. This is deliberately simple - there is no database, and a container restart loses all history. That's an accepted limitation for a cyber range demo, and one worth naming explicitly if

this were ever pitched as a production product (see Section 11).

4.6 `main_runner.py` - the entrypoint

Starts the dashboard API in a background thread, loads the two pickled models, then starts one capture thread per interface. Simple by design - almost all the real logic lives in the modules it imports.

4.7 `calibrate_tool.py` - real-traffic calibration helper

A standalone script, not part of the live pipeline, written to solve a specific problem: real attack tools (nmap, hping3) produce genuinely different flow statistics than the hand-crafted Scapy packets the model was originally calibrated against. Run inside the sensor container while a real tool is firing from the attacker node, it captures the resulting NFStream feature vector and prints it in a format ready to paste into `train_model.py`'s injection list - closing the loop between "what does this tool actually produce on the wire" and "what does the model need to be shown to recognize it."

5. The Triage & Resolution Flow

This is the human-facing heart of the system: what actually happens, step by step, between "a flow looks suspicious" and "something gets blocked, deferred, or ignored."

5.1 Entry gating

Before the LangGraph pipeline runs at all, `trigger_swarm()` checks two things and can exit early on either: is this source IP already blocked (skip - it's already handled), and has this specific source IP triggered the pipeline within the last 30 seconds (skip - per-source cooldown, to avoid flooding the Gemini API during a sustained attack). The cooldown is deliberately *per source IP*, not global - an earlier version used one global cooldown timer, which meant a second, unrelated attacker's alert could be silently swallowed just because someone else had tripped the cooldown moments earlier. That was a real detection gap, fixed by keying the cooldown dictionary on source IP.

5.2 Triage & Context

Both nodes have a dry-run branch (a canned f-string, used by default so testing doesn't burn API quota) and a real branch that calls Gemini. Context's real branch is agentic: rather than Python fetching the asset registry lookup and handing the result to the model, a `create_react_agent` is given the `lookup_asset_tool` and decides for itself to call it - and the prompt is deliberately neutral about whether containment is warranted, since that decision belongs to Response alone.

5.3 Response - the decision tree

Response computes the target asset's registered criticality and sets `requires_approval` for anything Critical or High. From there:

Mode	Requires approval?	Outcome
Dry-run	No	Calls <code>block_ip()</code> directly
Dry-run	Yes	Calls <code>request_approval()</code> - queued, no action
Real (agent calls the tool)	Either	<code>_decide_and_contain()</code> re-checks and routes to block or approval
Real (agent doesn't call the tool)	-	Status Bypassed, nothing happens
Real (LLM raises an exception)	Either	Fails <i>closed</i> : still respects approval requirement, otherwise force-blocks

Why the agentic branch checks a structured tool call instead of parsing text: an earlier version decided whether to block by checking `"BLOCK" in decision.upper()` against free-text model output. That is a real, demonstrated failure mode: a response like *"We should NOT BLOCK this IP, it's low severity"* still contains the substring `BLOCK` and would have triggered containment anyway - the exact opposite of what the sentence says. The fix binds `block_ip_tool` as a real LangChain tool and only acts when the model's structured `tool_calls` field actually names it; a regression test scripted a fake model returning exactly that sentence and confirmed the IP does *not* get blocked.

5.4 What happens after a decision

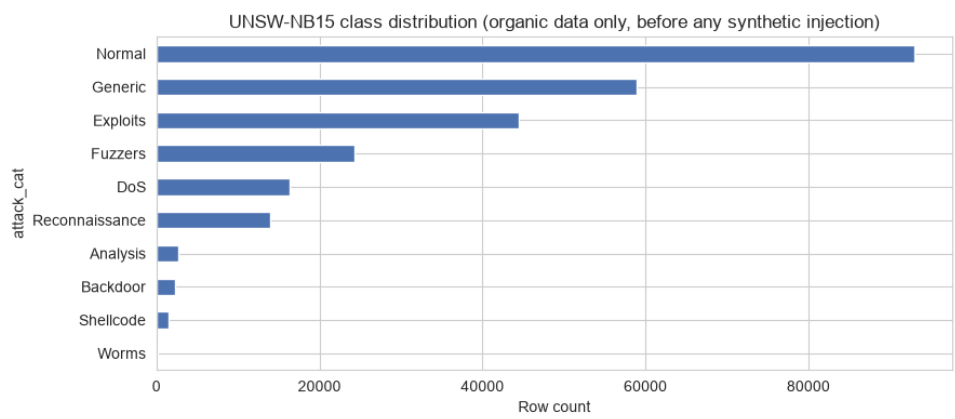
- **Block executes** → recorded with a `blocked_at/expires_at` pair; a background thread sweeps every 30 seconds and auto-unblocks anything past its TTL (default one hour).
- **Queued for approval** → sits in the Triage Queue until a human clicks Contain (calls `block_ip()` for real) or Dismiss (discards it, logs a ``DENIED`` mitigation, never touches `iptables`).

Regardless of path, everything converges on the same `block_ip()` / `request_approval()` functions - this single choke point is where the guardrails in Section 8 actually live, so nothing can bypass them by taking a different code path through the pipeline.

6. The Machine Learning Pipeline

6.1 Dataset

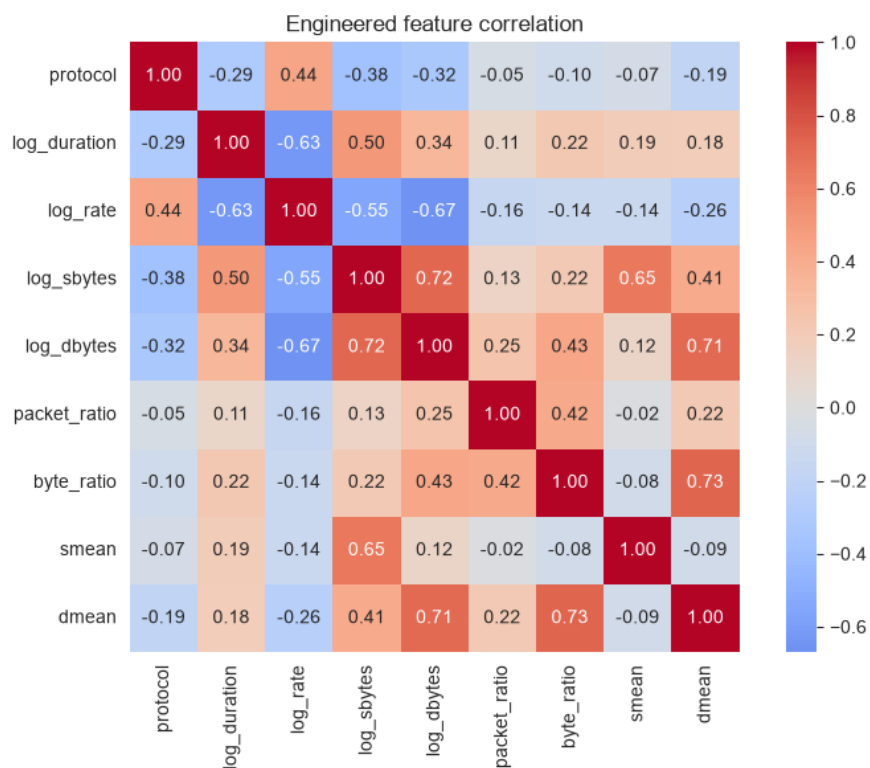
UNSW-NB15, a widely-used labeled intrusion detection dataset (257,673 rows across the provided train/test split, concatenated and re-split by this project for evaluation). Class distribution is heavily imbalanced - Normal and Generic attacks dominate, while Backdoor and worms are a tiny fraction, which is why the multi-class stage uses oversampling (below).



6.2 Feature engineering

NFStream's live capture doesn't expose the same ~40 columns UNSW-NB15 ships with, and a live volumetric flood can produce byte counts orders of magnitude past anything organic. Both models are trained on 9 robust, log-scaled features instead:

Feature	What it captures
protocol	TCP/UDP/ICMP mapped to IANA numbers (6/17/1), else 0
log_duration, log_rate, log_sbytes, log_dbytes	log1p-transformed volume/timing stats, to keep an extreme flood from blowing out the decision boundary
packet_ratio, byte_ratio	destination/source ratios - captures behavioral shape (a port sweep sends far more than it receives) independent of raw volume
smean, dmean	mean packet size per direction - stable regardless of flow duration or packet count



6.3 Synthetic signature calibration

The attacker's forged packets need to land inside a class the model recognizes, so exact profile constants (Section 3.1's table) are injected into training as anchor rows.

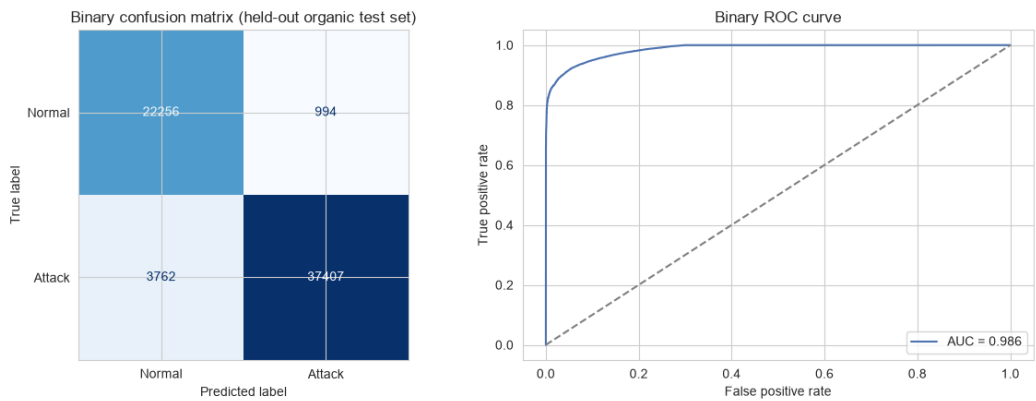
The overfitting mistake, found and fixed: the original approach injected each signature as 5,000 byte-identical copies - which teaches the model one exact *point* in feature space, not a *region*. Tested against realistic jittered variants of each signature (simulating what a real capture of the same attack actually looks like, with natural timing noise), that approach scored as low as **10% accuracy on Fuzzers**. The fix jitters each copy with Gaussian noise scaled to that feature's own natural spread in the organic data - 1,000 copies per class, 5% noise, a setting arrived at by sweeping several copy-count/noise combinations and picking the one that raised the weakest class's accuracy without regressing the others. That configuration scores 100% on DoS/Backdoor/Exploits, 87% on Reconnaissance, and 83% on Fuzzers under the same realistic-jitter test - a large, measured improvement, not a guess.

6.4 Training

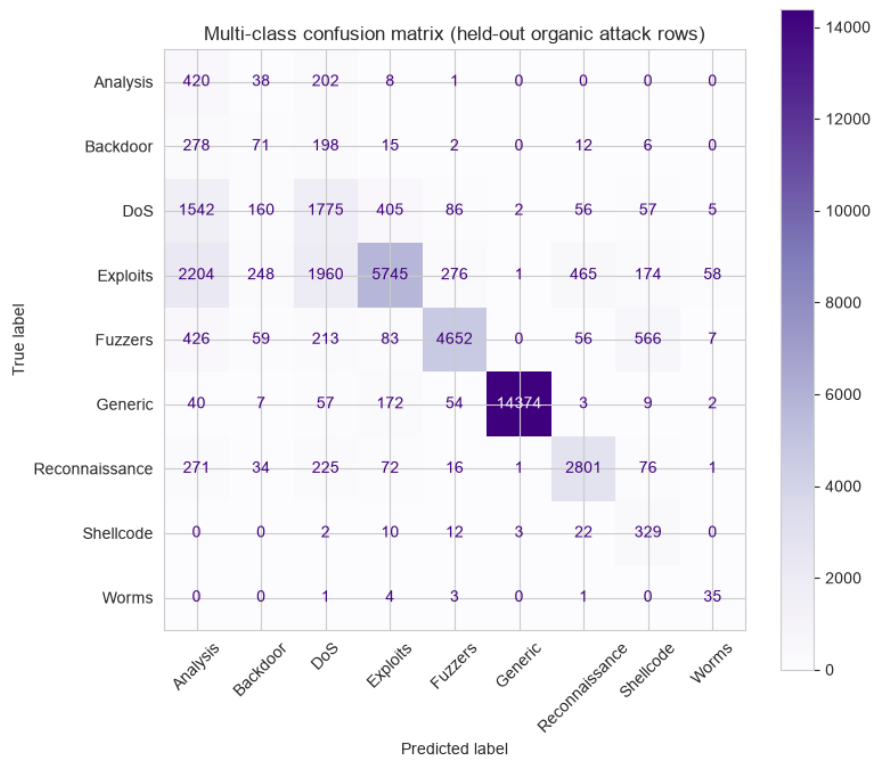
Two `RandomForestClassifiers`, `n_estimators=150`, `max_depth=15`, `class_weight='balanced'`. Stage 1 (binary) trains on the full organic+synthetic set. Stage 2 (multi-class) trains only on attack rows, oversampled with `RandomOverSampler` - not SMOTE, specifically to keep every training row 100% real/duplicated data rather than synthetically interpolated, matching the project's stated design principle of organic-data purity.

6.5 Evaluation

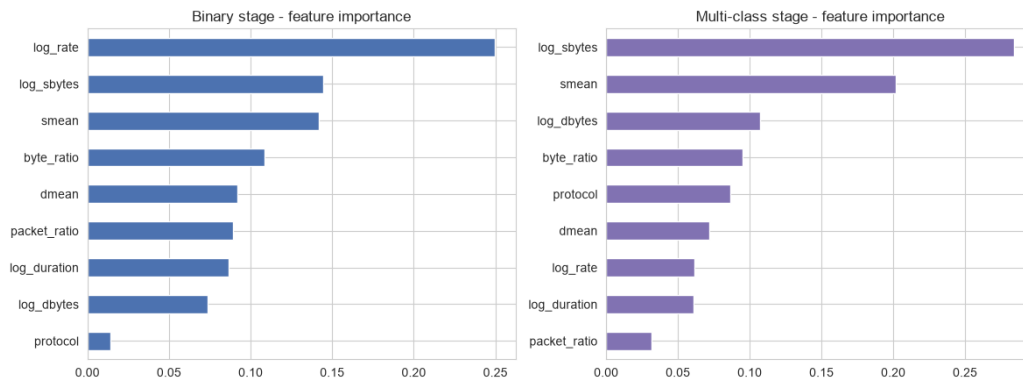
Binary classifier (held-out organic test set)



Multi-class classifier

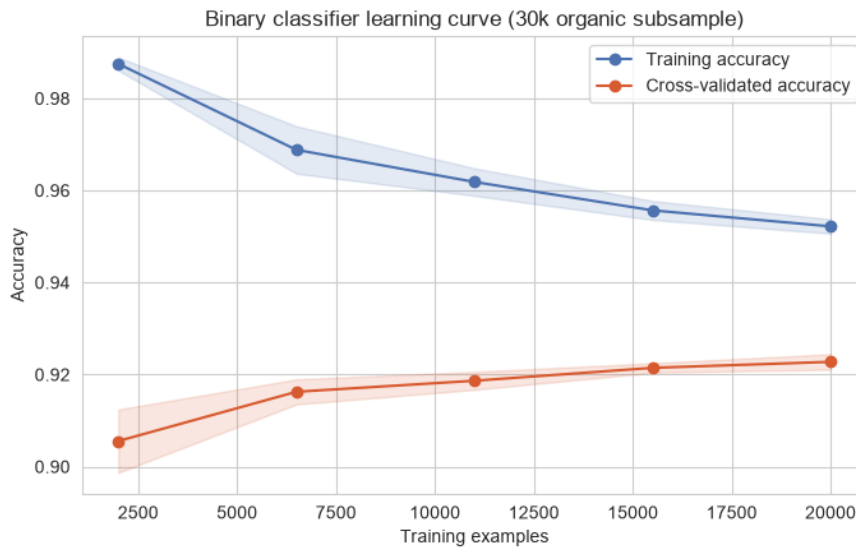


Feature importance

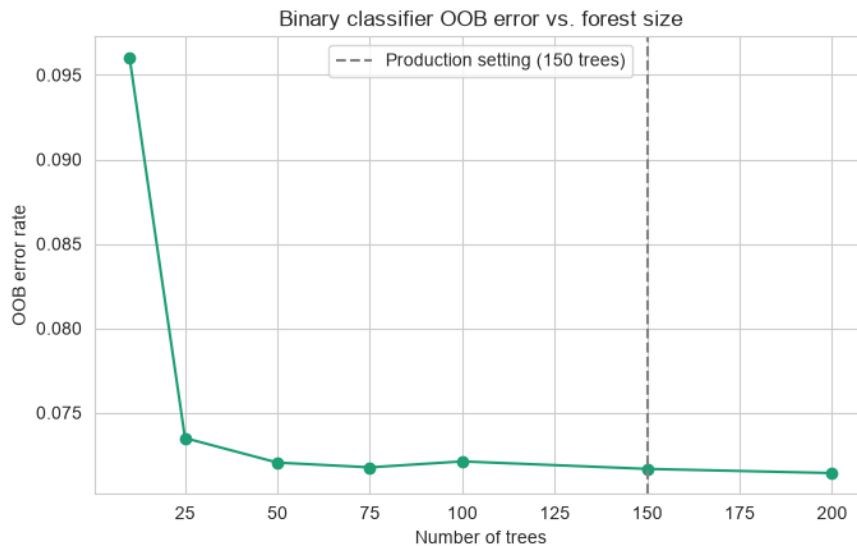


6.6 Learning curves and forest-size convergence

Random Forests don't have an epoch loss curve the way a neural net does; the two standard analogous diagnostics are a train-size learning curve and out-of-bag (OOB) error vs. forest size.



The gap between training and cross-validated accuracy stays small (about 0.03) and doesn't widen at larger training sizes - the model isn't meaningfully overfitting, and collecting more organic data would likely yield only marginal further gains.



OOB error is 0.0717 at 150 trees (the production setting) versus 0.0714 at 200 - the forest has essentially converged by 150 trees, confirming that hyperparameter as a reasoned choice rather than an arbitrary one.

6.7 Known limitation: multi-class confusion

Fuzzers, Exploits, and Reconnaissance are frequently confused with each other in the multi-class stage - all three are TCP attacks with similar packet counts and mean packet sizes, sitting in adjacent regions of the current 9-feature space. This was confirmed live: launching a Fuzzer attack was observed to be logged as `Exploits` in the dashboard on repeated tests. Critically, the **binary** call - Attack vs. Normal, the one that actually gates the mitigation pipeline - remained correct every time; only the cosmetic sub-label was wrong. Properly separating these three would need new discriminating features (e.g. inter-arrival time variance, TCP flag distributions), not further tuning of the injection parameters - a good example of "the fix that would actually work" not being "more of the same fix that already helped."

7. The Dashboard UI

A React + TypeScript single-page app (Vite dev server, dark theme), polling the sensor's API every 1.5 seconds. Recently reorganized from a 3-tab layout (Overview / AI Reports / Debug Logs) into 6 purpose-built views, specifically to give the human-in-the-loop approval flow a real interface - it previously had none, despite the backend supporting it fully.

7.1 Command Center

The landing view. A metrics strip (Network Status, Pending Review, Active Blocks, Mean Time to Containment - the last one computed live by matching mitigation timestamps back to the alert that caused them, not a placeholder), the LangGraph pipeline tracker (four nodes, hover for the full markdown report), Live Threat Intelligence (recent alerts), a network anomaly area chart, and a "Guardrail Activity" panel - horizontal bars showing how many flows were Contained, sent to Pending review, Denied, Refused by the allow-list, or Rate limited, so the safety systems are visible instead of buried in logs.

7.2 Triage Queue

The new human-in-the-loop surface. A live badge on the nav item shows the pending count. Each card shows the source/destination pair, target criticality, attack type and confidence, the triage report, and how long it's been waiting, with **Contain** and **Dismiss** buttons wired directly to `/api/approve` and `/api/deny`. Verified end to end: triggering a DoS attack against the High-criticality `hr-portal` produces a real card here; clicking Contain both clears the card and actually blocks the IP (confirmed via the active-blocks API, not just the UI updating).

7.3 Mitigations

A manual block form (any IP, subject to the same allow-list and rate limit as the automated path), a live table of active blocks with a real countdown to TTL expiry and an Unblock button, and a full mitigation history table with color-coded status badges.

7.4 AI Reports

An archive of every Triage/Context report generated, newest first, rendered as markdown cards.

7.5 Simulation Control

The attacker panel, deliberately moved out of the main command view into its own page - a real SOC operator's dashboard wouldn't have a "launch attack" panel front and center, since that's a testing capability specific to this cyber range, not a feature of the product itself.

7.6 Debug Logs

A filterable tail of `system.log` plus the attacker node's own log, refreshed on the same polling cycle.

7.7 Resilience: the Error Boundary

A real production bug, found live and fixed: LangChain's message .content field isn't always a plain string - some Gemini response shapes (grounding/citation metadata attached) return a list of content blocks like `[{'type': 'text', 'text': '...', 'extras': {...}}]` instead. That object shape got stored as an event message and eventually reached `<ReactMarkdown>{message}</ReactMarkdown>` - React cannot render an object as a child, throws, and with no error boundary anywhere in the tree, the *entire app* unmounted, leaving only the dark page background: a full black screen, reported during actual use. Fixed two ways: a `_extract_text()` helper in the backend coerces every LLM-derived message to a flat string before it's ever stored (the real fix), and a React `ErrorBoundary` now wraps each tab's content, keyed by the active tab, so if anything ever throws again - this bug or a future one - only that view shows a "try again" fallback instead of the whole app going blank.

8. Guardrails & Safety Mechanisms

These five mechanisms are what separate "an LLM decides to run iptables commands" from something defensible as a safety-conscious design. Understanding them together, as a system, matters more than any one in isolation.

8.1 Protected allow-list

A hard-coded set of addresses the agent can never block, regardless of what any alert or model output says: the sensor's own gateway addresses, plus *every IP in the registered asset database*, extensible via a `PROTECTED_IPS` environment variable.

Why "every registered asset," not just gateways - a real incident during testing: because `eth0` and `eth1` each run an independent `NFStreamer` capturing the same physical conversation, one of them can observe a UDP flow with source and destination reversed (hr-portal's ICMP "port unreachable" reply looks, from that interface's vantage point, like hr-portal initiating a new flow toward the attacker). This was observed live: the agent attempted to block `10.5.2.30` (hr-portal itself) instead of the real attacker. The fix - extending the allow-list to cover every registered asset, not just the two gateway addresses - was then confirmed live to correctly refuse the exact same corrupted detection on a repeat run, logging `REFUSED: 10.5.2.30 is on the protected allow-list` instead of severing a legitimate internal asset.

8.2 Rate limiting

A rolling 60-second window capped at `MAX_BLOCKS_PER_WINDOW` (default 20, environment configurable). Once exceeded, further blocks are refused and logged for manual review rather than executed. Deliberately set generously rather than strictly, since the dashboard's own "Randomize Source IP" DoS demo is designed to show many attacker IPs getting blocked in quick succession - a tight limit would break that showcase; the guardrail exists for genuine runaway/spoofed floods, not to interfere with the intended demo.

8.3 Tiered autonomy (human-in-the-loop approval)

Attacks targeting a `Critical` or `High` criticality asset are queued for a human decision instead of auto-resolving. The reasoning: a false-positive block has a cost proportional to what it disrupts, so the assets where a wrong block would do the most damage are exactly the ones where a human should be in the loop before anything executes.

8.4 Block TTL expiry

Every automated block carries an expiry (default one hour), swept by a background thread every 30 seconds. Containment is a hypothesis, not a permanent verdict - this ensures a false positive doesn't stay blocked forever without being revisited, without needing a full environment reset to undo it.

8.5 Per-source cooldown

30 seconds between LangGraph invocations for the same source IP, to avoid flooding the Gemini API during a sustained attack - keyed per source, not globally, for the reason explained in Section 5.1.

9. Debugging Journal & Lessons Learned

This section exists because a study document is more useful if it shows the actual debugging process, not just the finished state - these are real bugs, found through hands-on testing in a live Docker environment, not hypothetical.

9.1 The reversed-detection safety bug

Covered in full in Section 8.1. The short version: dual-interface capture of the same physical conversation can yield a flow with source and destination swapped, and the agent tried to block a registered internal asset instead of the real attacker. Caught live, fixed by broadening the allow-list, confirmed fixed on a repeat of the exact same scenario.

9.2 NFStream timing variability

Throughout testing, the delay between traffic actually crossing the wire and NFStream reporting a completed flow ranged from near-instant to 90+ seconds, unpredictably, even for identical traffic patterns. This is an environment characteristic worth knowing before assuming "no alert yet" means "not detected" - in one specific test, traffic confirmed present via ``tcpdump`` took over 60 seconds longer than expected to surface as a classified flow. Practical implication for live demos: let an attack run at least 5-10 seconds before stopping it, so NFStream's idle timeout has a reliable chance to close and flush the flow.

9.3 Real attack tool calibration

When asked why real ``nmap`` traffic wasn't being detected, the actual root causes (found by live packet capture and log inspection, not guessed) were: (1) a genuine multi-port scan fragments into many tiny flows the per-flow classifier can't see the pattern across - solved by the port-scan correlator (Section 4.1); and (2) ``hping3``'s default behavior increments the source port on every packet, which splits what should be one aggregated flow into many single-packet flows - solved by adding ``-k -s`` to pin it. Both fixes were verified live against the running sensor before being considered done.

9.4 The black-screen React crash

Covered in full in Section 7.7. Confirmed via the user's actual browser console, root-caused to a non-string LLM content shape, fixed at the source (backend coercion) and backstopped (frontend error boundary), and verified by inspecting every live flow-event message's Python type after redeploying both fixes - all confirmed `str`, including a genuine successfully-returned Gemini markdown report.

10. API Reference

Endpoint	Method	Purpose
/api/alerts	GET	Last 100 ML/correlator-detected alerts
/api/mitigations	GET	Last 100 mitigation actions (blocks, refusals, denials, etc.)
/api/flow	GET	Last 50 LangGraph pipeline step events
/api/pending_approvals	GET	Everything currently queued for human review
/api/active_blocks	GET	Currently blocked IPs with blocked_at / expires_at
/api/approve	POST	`{ip_address}` - approve a pending containment
/api/deny	POST	`{ip_address}` - dismiss a pending containment
/api/trigger	POST	`{ip_address}` - manual block, subject to the same guardrails
/api/unblock	POST	`{ip_address}` - manual unblock
/api/simulate_attack	POST	Proxies to the attacker node's `/attack`
/api/stop_attack	POST	Proxies to the attacker node's `/stop_attack`
/api/reset	POST	Flushes iptables, clears all in-memory state, stops all attacks
/api/logs, /api/attacker_logs	GET	Raw log tails for the Debug Logs view
/api/clear_logs	POST	`{category}` - clears a log category

11. Known Limitations & Future Work

- **Multi-class sub-labeling** among Fuzzers/Exploits/Reconnaissance remains unreliable (Section 6.7) - would need new features, not more calibration tuning.
- **No persistence** - all state is in-memory; a container restart loses history. Fine for a demo, disqualifying for a real product without a database.
- **No authentication** on any API endpoint, and CORS is fully open - acceptable for a localhost cyber range, not for anything exposed beyond it.
- **Free-tier Gemini quota** (20 requests/day at time of testing) means real-LLM mode exhausts quickly; ``LLM_DRY_RUN=True`` is the practical default for repeated testing.
- **Real-tool coverage** is 2 of 5 attack types (DoS, Port Sweep); Exploits/Fuzzers/Backdoors remain synthetic-only, by measured choice, not oversight.
- **NFStream flow-completion latency** is unpredictable in this environment (Section 9.2) - worth knowing before assuming a lack of alert means a lack of detection.

For a fuller "what would you build next" discussion (mitigation connectors beyond raw iptables, real persistence, multi-tenant auth, structured LLM output schemas, etc.), see the product-roadmap discussion captured in this project's working session history.

12. Appendix

A. File manifest

Path	Lines	Purpose
soc_brain/live_sensor.py	~205	Packet capture, feature extraction, ML inference, port-scan correlator
soc_brain/orchestrator.py	~424	LangGraph pipeline, agent tools, all guardrails
soc_brain/dashboard_api.py	~202	Flask REST API
soc_brain/event_store.py	~58	Thread-safe in-memory state
soc_brain/train_model.py	~208	Data engineering, feature engineering, training
soc_brain/calibrate_tool.py	~82	Real-traffic calibration helper
soc_brain/main_runner.py	~36	Entrypoint
soc_brain/ml_training_notebook.ipynb	-	Executed, Kaggle-style training/evaluation notebook
attacker/attacker_api.py	~219	Synthetic + real attack engines
dashboard-ui/src/App.tsx	~766	All 6 dashboard views
dashboard-ui/src/ErrorBoundary.tsx	~50	Render-crash containment
dashboard-ui/src/index.css	~460	Design system / styling
docker-compose.yml	~130	Full service/network topology

B. Key environment variables

Variable	Default	Effect
GEMINI_API_KEY	(required for real mode)	Auth for Gemini calls
LLM_DRY_RUN	True	Canned responses vs. real Gemini calls
PROTECTED_IPS	gateways + registered assets	Extra comma-separated allow-list entries

MAX_BLOCKS_PER_WINDOW	20	Rate-limit cap per 60s window
BLOCK_TTL_SECONDS	3600	Auto-unblock delay

C. Common commands

```
docker compose up --build -d          # start everything
docker compose up --build -d dashboard-ui # rebuild frontend (build-time COPY, no live mount)
docker compose restart soc-sensor-brain  # apply backend code changes (live-mounted)
curl -X POST http://localhost:5000/api/reset # clear all state, flush iptables
```